ProEm: Verification Report

Design frozen at ISA v0.7.1

Paul Vincent Bezzina

13 October 2026

Abstract

The architecture is frozen at ISA v0.7. This report covers the checks run on that design before hardening: lint, synthesis to the IHP sg13cmos5l cells, gate-level simulation, a second simulator, an independent decode of the standards waveforms with sigrok, line coverage, and a formal proof of the line codec's entry accounting. Two real bugs came out of it, both in the line codec, both fixed with small local changes; nothing in the instruction set changed. The design is released as v0.7.1. What is left to do needs the Tiny Tapeout flow (placement, routing and timing with wires) or real hardware.

1 Summary

Check	What was run	Result
Freeze	v0.7 tagged as the baseline; the pulse-width work started on top of it was discarded	done
Lint	Verilator 5.020 `-Wall` on the whole design; 26 warnings, each reviewed; 13 waivers recorded in `lint/waivers.vlt`	**pass**
Synthesis	Yosys and ABC to `sg13cmos5l` cells, typical and slow corners (`scripts/synth_ihp.sh`)	35 % of the 6x4 tile
Gate level	The cocotb suite on the synthesised netlist with the IHP cell models (Icarus)	**pass** 6/6
Second simulator	Every pin-level test rebuilt on Verilator instead of CXXRTL	**pass**
Independent decode	30 waveforms from the standards tests decoded by sigrok's protocol decoders (`scripts/sigrok_check.py`)	**pass** 30/30
Coverage	Verilator line and branch coverage over the whole regression (`scripts/coverage.sh`)	98.7 %
New directed test	SPI controller in all four modes, both BRANCH rules, NOP (`sim/t_modes.cpp`)	bug, **fixed**
Formal	Unbounded proof of the line codec's entry accounting and output enables (`formal/line_codec.sby`)	bug, **fixed**
Regression	Full `scripts/run.sh`, cocotb, all six formal proofs, on the fixed RTL	**pass**

2 Bugs found and fixed

2.1 A lost `tx_done` after a cancel (found by the formal proof)

The codec answers every entry it accepts with exactly one `tx_done`; the frame chain counts entries in flight with it and only lets read blocks listen when the count is zero. When a frame is aborted,

`tx_cancel` throws away the entry being sent and the one waiting, and `dropCount` pays back one `tx_done` for each over the next two clocks.

The proof found two ways to lose one of those pulses:

- a second `tx_cancel` within two clocks overwrote `dropCount` instead of adding to it;
- a new entry accepted straight after the cancel could finish (a zero-length entry is answered at once) on the same clock as a pending drop pulse, and the two merged into one.

Either way the chain's count stays at one forever, `line_free` never returns, and the channel stops receiving until `CLEAR`. Whether the frame chain can produce those exact sequences was not proven, but nothing in the codec prevented them.

The fix is two lines in `line_codec.sv`: a cancel adds to `dropCount` (less the pulse being sent that clock), and the state machine waits in `T_IDLE` while `dropCount` is not zero, so no other `tx_done` can fall on the same clock. The cost is at most two clocks after an abort. The unfixed RTL fails the proof in 8 steps; the fixed RTL passes it unbounded.

2.2 No hold time in SPI modes 1 and 3 (found by the coverage gap)

Coverage showed that the generated-clock path for a trailing-edge sample had never run: every test used SPI mode 0, I^2C or MDIO. A new test drove the controller in all four modes into a device model that samples MOSI on its edge with no hold margin.

In modes 1 and 3 (CPHA 1) the chip changed the data pin at the start of each bit, which is the same system clock on which the clock returns to idle, the device's sampling edge. The model read the next bit every time (`A55A` came back as `4AB4`). A real device would see a hold time of about 0 ns plus whatever skew the board adds, so the result would depend on the board.

The fix holds the previous bit (`holdLevel`) through phase A when the codec generates the clock, samples on the trailing edge and is not in I^2C mode. The data now changes on the leading edge and is steady for a whole half period on either side of the sampling edge. Modes 0 and 2, I^2C, MDIO, SD card and 3-wire SPI do not take this path and are unchanged. All four modes now pass at 3 and 6 clocks per half period.

Two smaller items were found on the way. `docs/line_codec.tex` said modes 0 and 3 sample on the leading edge; it is modes 0 and 2, and the RTL was already right. `stuffBit`, a register nothing read, was removed.

3 Lint

```
bash scripts/lint.sh
lint clean: verilator -Wall, 13 reviewed waivers
```

The 26 warnings fall into four groups, all harmless:

- width warnings where an integer parameter or a 1-bit increment is extended on purpose;
- per-channel status outputs (`complete`, `abort`, `timeout` and others) that the top level does not route, the reserved `OUT` bit 0, and the two fraction bits of `blankSum`;
- block 0 inputs of `start_router`, which are always 1;
- `UNOPTFLAT` on `start_router`'s carry vector: a ripple from bit to bit, not a loop. Yosys `check -assert` confirms there is no combinational loop after synthesis.

The inputs are synchronised before use: the eight pins through three flip-flops and the host UART input through three. The one design point worth knowing is that each channel's reset is `rst_n` AND

NOT `clear`, where `clear` is a registered one-clock pulse. It is glitch free and releases just after a clock edge, so the flow's recovery and removal checks should show it clean.

4 Synthesis to the IHP cells

`scripts/synth_ihp.sh` maps the design the way LibreLane does (the same mux maps and excluded cells from the PDK's `librelane` directory) and reports area and logic depth. It needs `PDK_ROOT` pointing at an IHP-Open-PDK checkout; the run below used revision `2bbec755`, the one the Tiny Tapeout tools pin.

Measure	Value
Standard cells	19 721
of which flip-flops (`dfrbpq_1`)	2 666
Cell area	323 385 µm^2
6x4 tile (1289.28 × 710.64 µm)	916 214 µm^2
Utilisation before placement	35 %
Longest register-to-register logic, typical 1.20 V 25 °C	3.8 ns
Same, slow 1.08 V 125 °C	5.9 ns
Clock period (50 MHz)	20 ns

The delays are ABC's estimate with no wires, so they are a lower bound. An older ABC flow gave 10.0 ns at the typical corner for the same logic, which is a reasonable pessimistic figure. The flow's sign-off corner is typical (`TIMING_VIOLATION_CORNERS = *typ*`), with 0.25 ns uncertainty, 5 % derating and 20 % input and output delay. Even with wiring adding half again, there is a wide margin. ABC reports the longest path starting at the MAIN bit counter of a channel. The 35 % utilisation leaves room for the buffers, clock tree and fillers the flow adds; earlier documents quoted a cell budget from generic synthesis, and this figure replaces it.

5 Gate-level simulation

The cocotb suite (`test/test.py`) was run against the synthesised netlist with `GATES=yes`, using `sg13cmos5l_stdcell.v` with `FUNCTIONAL`: autobaud and errors, UART loopback, SPI loopback, I^{2}C target loopback, UART at 0x0000 and the CRC append and check. All 6 pass. This netlist comes from synthesis, not from placement and routing; the `gl_test` job in the `gds` workflow repeats the run on the routed netlist.

6 A second simulator

Every pin-level test was compiled a second time against a Verilator model through a small adapter (`sim/verilator/emu_sim.h`), without changing the tests. After one testbench fix, every test passes on both simulators. The fix: Verilator starts `rst_n` at 0, so driving it to 0 is not a falling edge and the asynchronous reset never fired. The adapter now starts `rst_n` high, as a real power-on does. CXXRTL and the gate-level cell models treat the reset as a level, so they were not affected, and neither is the chip.

7 Independent decode with sigrok

The C++ device models and the RTL were written by the same person, so a misreading of a standard could pass both. `scripts/sigrok_check.py` records the pins of each standards test as a waveform

(`PROEM_VCD`), decodes it with sigrok's protocol decoders, and checks the bytes against values computed separately (CRC-16 Modbus, NMEA checksum, LIN checksum and PID, SMBus PEC). Both directions are checked: what the chip sends and what the models send to it.

Decoder	Checked
`uart`	RS-485, MIDI, NMEA 0183, PROFIBUS (8E1), Modbus RTU with CRC, DMX512 slots, LIN frame, K-Line request and response, SDI-12 (7E1, inverted), each in both directions where the test has both
`dmx512`	Break, mark after break, start code
`i2c`	SMBus Write Word and Read Word with PEC
`mdio`	Clause 22 write and two reads
`spi`	SD card CMD0 and CMD8 with the chip's CRC-7; 3-wire SPI write and read
`dali`	Forward frame; both backward frames
`can`	CAN 2.0B extended frame, PGN 127250, eight data bytes
`ir_rc5`	Address 5, command 0x35

All 30 checks pass. Two decoder quirks needed working around, and neither is a ProEm problem. sigrok's `dmx512` decoder loses byte alignment when a slot's start bit follows the second stop bit with no gap (legal in DMX512), so the slots are decoded with `uart` at 250 kbit/s 8N2 and the break and start code with `dmx512`. The `dali` decoder reports one backward frame per capture, so the second frame is decoded from a window that starts just before it.

8 Coverage

```
bash scripts/coverage.sh
Total coverage (1310/1327) 98.00%
```

Of the 17 points never reached, 7 are `default` branches of complete case statements, which cannot be reached. The other 10:

- the Manchester receive counter saturating at $2^{22}-1$ clocks (more than 84 ms of idle at 50 MHz);
- a wait entry that starts while a stuff bit is due;
- the reply step skipped because its condition is false (for example, a reply set to "only on good CRC" after a bad frame);
- the return path serving global flags right after channel B while channel B is idle.

None of them changes how the chip behaves for a standard in the guide. The first coverage run also flagged BRANCH rule 1, SPI modes 1 to 3 and NOP; `t_modes` now covers them, and that is how the hold-time bug above was found.

9 Formal proof of the line codec

`formal/line_codec_formal.sv` wraps the unchanged codec with a reference count of accepted entries and proves, with every line, clock and output configuration left free and allowed to change at any time:

- `tx_done` never pulses with nothing outstanding, and at most four entries are ever outstanding;
- once the select output has been inactive for three clocks, every accepted entry has had exactly one `tx_done`, including entries dropped by any pattern of cancels;

- with `DRIVE` = `00` no output enable is ever set; with open drain the data and clock outputs only pull low; the clock output is only enabled while the codec generates the clock.

`abc pdr` proves this unbounded in about 20 s; the cover task reaches a double drop, a Manchester entry and an I^2C entry. It joins the five earlier proofs: both FIFOs, the UART transmitter, the branch unit and the CRC unit.

10 What is left

- **Timing with wires.** Push to GitHub and let the `gds` workflow harden the design. Check the typical-corner setup and hold slack, the recovery and removal checks on the channel resets, the utilisation, and the routing congestion report. This is the one result that could send the design back to the RTL.
- **Routed gate level.** The `gl_test` job runs the cocotb suite on the routed netlist. It should match the result above.
- **Hardware.** An FPGA build against real parts (an I^2C sensor, a CAN transceiver, a USB-UART adapter), with a logic analyser running the same sigrok decoders as `sigrok_check.py`.
- **Limits that stay.** A generated clock needs `DIV` ≥ 4 (as the ISA says), because input pins arrive three clocks late. `t_modes` runs at 3 to show the margin; at 2, CPHA 1 reads fail as expected.

11 Running it all

```
bash scripts/lint.sh                      # Verilator -Wall
bash scripts/run.sh                       # every pin-level test, then synthesis
cd test && make && cd ..                  # cocotb, RTL
bash scripts/formal.sh                    # six proofs (OSS CAD Suite)
bash scripts/coverage.sh                  # Verilator coverage, report in build/cov
python3 scripts/sigrok_check.py           # needs sigrok-cli and build/t_standards
PDK_ROOT=~/IHP-Open-PDK bash scripts/synth_ihp.sh typ_1p20V_25C
```